



Designing fraud-prevention systems that keep analysts in the loop

José P. Barrantes

IBM Cloud, Fraud Science and Engineering

jospablo777@gmail.com

Companion article for the [Nerdearla Chile 2026](#) talk · Santiago, April 17, 2026

Abstract. This article extends the Nerdearla Chile 2026 talk “Fraud prevention, machine learning, and design patterns: keep your analysts in the loop.” We argue that fraud prevention is best treated as an **ML-enabled software-architecture problem** rather than as a classification problem alone. In real operations, rules, machine learning, operational policy, analyst work, and platform engineering all shape outcomes, and judging the system by model performance alone hides its socio-technical nature.

We propose a design pattern for **ML-enabled Human-in-the-Loop (HIL) Triage**: rules handle explicit cases, machine learning scores and prioritizes, a policy layer routes cases and selects actions, and expert analysts take the ambiguous cases and return structured feedback. We connect this architecture to the literature on software architecture for ML systems, human-in-the-loop design, design patterns for AI-based systems, learning to defer, alert prioritization, reliable machine learning, and platform engineering. We also situate the proposal within the wider human-in-the-loop machine-learning literature, where humans participate as reviewers, collaborators, or teachers rather than only as post hoc annotators.

1 Introduction

The conference talk that accompanies this repository was intentionally concise. A short slot works well for the core intuition, but it leaves no room for the software-architecture background, the design-pattern framing, the operational consequences, or the connections to MLOps and platform engineering. This article fills that gap.

The central thesis is that robust fraud-prevention systems are hybrid systems. They rarely succeed as purely manual workflows, since traffic volume and

Hybrid system:
Combines deterministic rules, ML scoring, decision policy, and expert human judgment rather than relying on any single component.

attack speed make full manual review infeasible. They also rarely succeed as purely automatic systems, because high-stakes decisions have to absorb ambiguity, changing attacker behavior, incomplete labels, asymmetric costs, and governance constraints. The most resilient architectures combine deterministic controls, statistical scoring, decision policy, and expert human judgment.

That claim is architectural, not just organizational. Production machine-learning systems already require supporting software, data pipelines, monitoring, deployment structures, and cross-functional engineering practices (Amershi et al., 2019a; Lewis et al., 2021b; Nazir et al., 2024; Sculley et al., 2015). In fraud prevention, the surrounding system also includes analysts, decision queues, escalation paths, structured review outcomes, and operational feedback loops. The unit of design is therefore the socio-technical system in which model outputs, human judgment, workflow, and institutional decision-making interact to convert signals into action, rather than a machine learning (ML) model in isolation (Dobbe and Wolters, 2024; Salwei and Carayon, 2022).

Socio-technical system: Behavior shaped by both technical components (models, pipelines, infrastructure) and human work (analyst review, organizational decisions, governance).

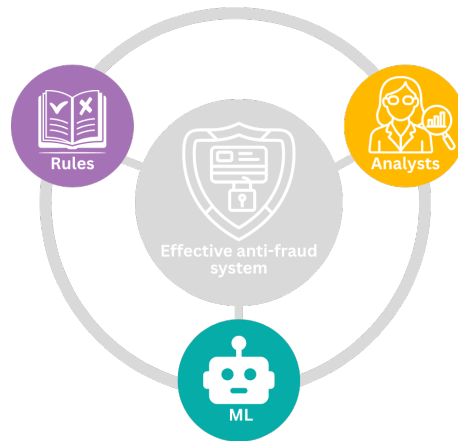


Figure 1. A conceptual overview of the pattern: effective fraud systems are built from the interaction of rules, machine learning, and expert analysts rather than any one component in isolation.

Figure 1 captures the argument in its simplest form. Fraud systems benefit from three complementary capabilities: rules encode explicit knowledge and guardrails, machine learning compresses heterogeneous signals into scores or rankings, and analysts contribute judgment, novelty detection, and feedback. None of the three dominates. The design work lies in how they interact.

2 Why fraud prevention becomes an architecture problem

Fraud prevention is not a stable prediction problem that you solve once and serve indefinitely. In practice, fraud systems tend to combine rules, supervised models, anomaly detection, and human investigation, because no single technique fully handles novelty, delayed labels, and operational

constraints. [Carcillo et al. \(2021\)](#) describe a typical fraud detection system as a multi-layer control structure that can include both automated and human-supervised components, while [Hernandez Aros et al. \(2024\)](#) show that the literature still relies on a wide mix of supervised, unsupervised, deep-learning, and other approaches rather than a single dominant recipe.

First, attackers adapt. [Bolton and Hand \(2002\)](#) already emphasized that fraud detection is a continuously evolving discipline: once a detection method becomes known, criminals change strategy. [Carcillo et al. \(2021\)](#) make the same point in the credit-card setting, where customer behavior changes over time and fraudsters adapt to the detection techniques themselves. [Lunghi et al. \(2023\)](#) extend this into the adversarial-learning literature, where fraud detection is treated as a security-sensitive environment shaped by concept drift, streaming constraints, limited observability, and hostile adaptation. Earlier reviews of anomaly detection in financial fraud reach similar conclusions about the need for adaptive, multi-method approaches ([Hilal et al., 2022](#)).

Second, labels are delayed and imperfect. [Carcillo et al. \(2021\)](#) note that fraud labels are usually known only a posteriori, either because a customer complains or because an investigation confirms the case, and that not all labels are available immediately. [Lunghi et al. \(2023\)](#) likewise describe real-world fraud detection as a delayed-feedback setting in which suspicious transactions are often analyzed by human investigators before a card is blocked. Beyond delay, the labels themselves are imperfect. Some fraudulent cases are never reported and remain wrongly labeled as legitimate, which means the training data is contaminated by undetected fraud ([Bolton and Hand, 2002](#); [Carcillo et al., 2021](#)). Fraud is also a heterogeneous phenomenon: although it is often treated as a binary classification problem, real fraud instances can correspond to different types of abuse, each with distinct patterns and indicators ([Bolton and Hand, 2002](#); [Hernandez Aros et al., 2024](#)). Collapsing that variety into a single positive class hides structure that could inform detection, triage, and investigation. The system therefore does more than predict a fixed ground truth; it acts inside the very environment that later generates the labels used for learning, evaluation, and retraining.

Third, error costs are asymmetric. [Bolton and Hand \(2002\)](#) describe fraud detection as a ranking and suspicion-scoring problem in which investigative attention must be concentrated on the most suspicious cases, because investigating everything is too expensive. They also stress that prevention and detection involve compromises between expense, inconvenience, and effectiveness. [Hernandez Aros et al. \(2024\)](#) similarly note that false fraud alarms, misclassification costs, and timely detection remain central practical difficulties in financial-fraud detection. What the system manages, then, is less a neutral statistical objective than a policy trade-off between missed harm and unnecessary friction.

Fourth, many decisions are time-sensitive. The value of fraud detection is a

Adversarial adaptation: Fraudsters change tactics once a defense becomes known, making detection a moving contest.

Label delay and imperfection: Ground truth is often confirmed only after complaints or investigations. Undetected fraud contaminates the legitimate class, and fraud itself is heterogeneous rather than a single category.

Asymmetric error costs: False negatives cause direct loss; false positives cause friction. Not interchangeable.

Time sensitivity: A correct decision made too late may still be operationally poor. The value of fraud detection is a function of time.

function of time: the sooner the fraud is detected, the less the potential cost of the event (Hilal et al., 2022). Bolton and Hand (2002) argue that fraud must be detected as quickly as possible once prevention has failed, and that suspicion scores should be ranked so investigation can focus on the most urgent or suspicious records. Lunghi et al. (2023) show that fraud detection often operates online and under streaming constraints, where timing affects both attacks and defenses. Carcillo et al. (2021) add that real fraud systems are layered and partially human-supervised, which means architecture determines which cases can be handled automatically, which must be escalated, and how scarce analyst attention is reserved for the ambiguous zone.

These properties help explain why the literature on ML-enabled software systems has moved from model-centric discussions toward more system-centric and architecture-centric thinking. Muccini and Vaidhyathan (2021) argue that ML-based systems require dedicated architecting practices beyond those of traditional software alone. Lewis et al. (2021b) likewise frame ML systems as end-to-end systems that must be developed, monitored, maintained, and evolved with architecture in mind, while Nazir et al. (2024) show that current architecting guidance increasingly focuses on lifecycle, infrastructure, quality, and integration concerns rather than only on the learned component.

3 From hidden technical debt to ML-enabled fraud systems

Sculley et al. (2015) argued that much of the real cost of ML systems lies outside the model itself. Data dependencies, feature extraction, configuration, serving, monitoring, and process tooling all create hidden technical debt. The insight has particular force in fraud prevention, where it exposes a tempting simplification: the fraud model is never the whole fraud system.

Hidden technical debt:
The real cost of production ML lies mostly outside the model: data pipelines, configuration, serving, monitoring, and glue code accumulate maintenance burden.

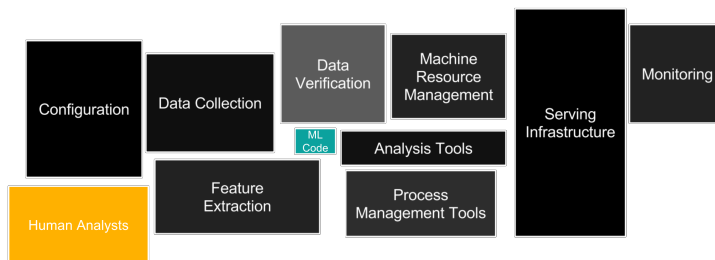


Figure 2. System anatomy for fraud operations: the model lives inside a wider production environment made of data, software, infrastructure, monitoring, and expert human work. Adapted from Sculley et al. (2015).

We adapt the system-anatomy perspective of Sculley et al. (2015) to fraud operations (Figure 2). It places the learned component inside a wider environment of data flows, serving infrastructure, process tools, monitoring, and human work. This matters because many operational failures appear at the boundaries:

a feature outage, a bad integration, a misleading analyst view, or a monitoring blind spot can be just as damaging as a weak model. [Lin and Ryaboy \(2013\)](#) reinforce the same lesson from large-scale data-mining infrastructure, where workflows, recurring jobs, and supporting data plumbing become first-class engineering concerns rather than background implementation details.

The same system view should shape the planning phase, not only post-deployment troubleshooting.

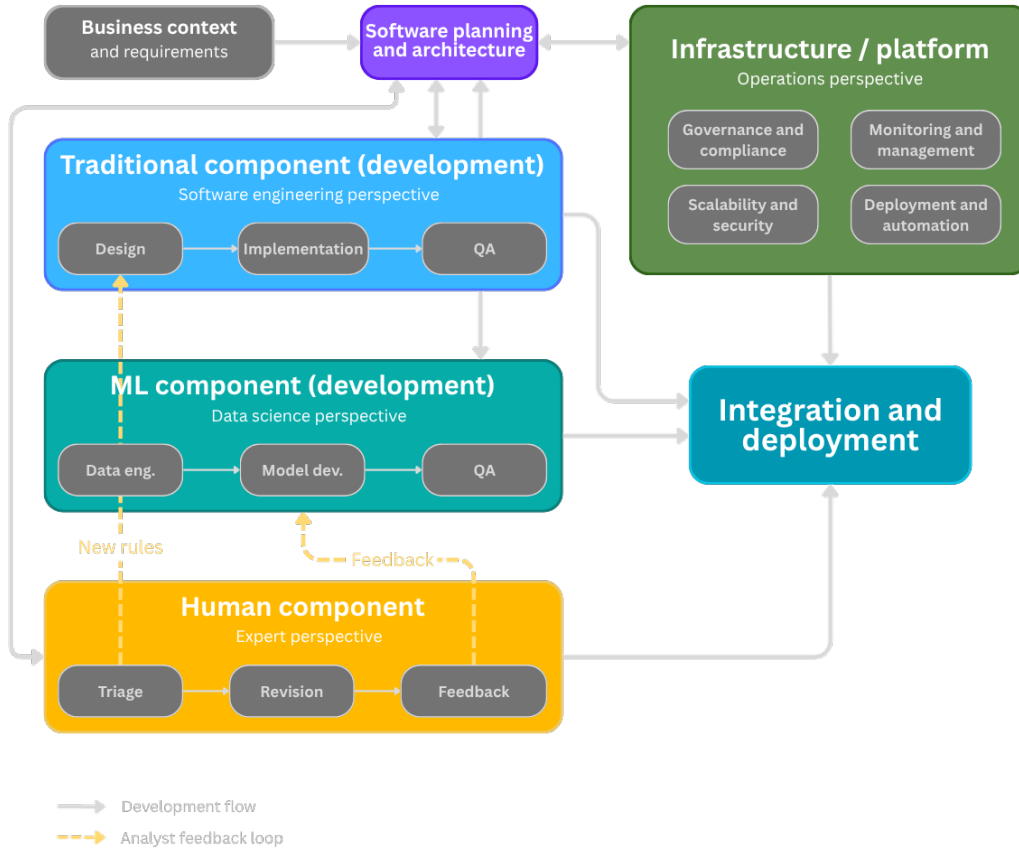


Figure 3. Planning view of an ML-enabled fraud system: business context, software architecture, traditional software, ML development, human expertise, and platform concerns should be co-designed. Adapted from [Lewis et al. \(2021b\)](#), [Andersen and Maalej \(2024\)](#), and [Kästner \(2025\)](#).

Figure 3 broadens the lens from runtime anatomy to planning. Business requirements, conventional software, ML development, expert review, and platform infrastructure should be designed together. [Lewis et al. \(2021b\)](#) treat software architecture as the central organizing activity for ML-enabled systems rather than a one-time diagram produced before implementation. In that view, architecture receives business context and system requirements, decomposes the system into distinct component lanes, and coordinates how those lanes will evolve together. The bidirectional arrows matter: development is not a one-way handoff. When teams hit problems in software engineering, model development, data pipelines, quality assurance, or operations, those issues flow back to architecture, where they can be discussed centrally and propagated

Co-architecting: The runtime system and model-development pipeline should be designed together, not as disconnected work streams.

across the rest of the system. Architecture helps prevent mismatch between teams that would otherwise design in parallel and drift apart in assumptions, constraints, and quality goals.

The rest of Figure 3 also helps clarify why ML systems must be planned as products rather than isolated models. The traditional component lane represents code, APIs, business logic, integrations, and interfaces. The ML lane represents model requirements, data engineering, model development, and quality assurance, and it produces the supporting data pipeline alongside the trained model. Lewis et al. (2021b) use this planning view to motivate co-architecting and co-versioning: the system side and the model-development side must be coordinated, and the deployed model must remain traceable to the data, parameters, and evaluation artifacts that produced it. The infrastructure and platform lane wraps the whole system with governance, monitoring, scalability, security, and deployment automation, all of which matter because ML components are data-dependent and degrade differently from conventional software. Our adaptation adds a human lane, which makes explicit something fraud operations often leave implicit: expert review is a component to design and integrate from the beginning, not an afterthought.

Co-versioning: Data, features, models, evaluation artifacts, and policy thresholds should be traceable together so post-incident analysis can reconstruct the full decision state.

4 Why we propose a design pattern

The architecture discussed here is more than a one-off diagram. We propose it as a reusable response to a recurring class of problems, which in design-pattern terms means specifying context, forces, solution, and consequences (Konieczny, 2025; Lakshmanan et al., 2020).

Design pattern: A reusable solution to a recurring problem in a specific context. Specifies context, forces, solution, and consequences.

Pattern literature for AI and ML systems is still maturing. Washizaki et al. (2020) documented recurring architecture and design patterns for machine-learning systems. Heiland et al. (2023) expanded the pattern repository for AI-based systems. Järvenpää et al. (2024) focused on reusable architectural tactics for ML-enabled systems. Cruz et al. (2023) reinforced the importance of architecture rationale and evaluation. Together, these works suggest that teams need more explicit and reusable architecture knowledge for ML-enabled systems.

ML architecture patterns: Washizaki et al. identified 33 ML patterns. Heiland et al. expanded to 70 AI-system patterns.

Andersen and Maalej (2024) make that point especially concrete for human-in-the-loop settings: they frame HIL as a software-engineering design space and compile a catalog of reusable HIL patterns spanning data preparation, training, operation, monitoring, and explanation. Their work matters here because it shows that human participation can itself be treated as a pattern concern rather than as an informal fallback around an otherwise automated model.

HIL patterns: Andersen & Maalej catalog reusable HIL patterns spanning data prep, training, operation, monitoring, and explanation.

This is the motivation for formalizing an ML-enabled HIL triage pattern for fraud, and the broader HIL-ML literature strengthens the move. Mosqueira-Rey et al. (2023) describe human-in-the-loop machine learning as a family of

HIL-ML family: Active learning, interactive ML, machine teaching. Differs in *how* and *when* human expertise enters the loop.

approaches, including active learning, interactive machine learning, and machine teaching, that differ in how human expertise enters and shapes the loop. Fraud operations are closer to that richer view than to the narrow image of humans appearing only after the system has already made a decision. Andersen and Maalej (2024) also help make the operational implication clearer: HIL extends past model training into deployed-system patterns such as recommendation support, active moderation, corrective feedback, continuous learning, and explanation. That operational orientation maps closely to fraud review queues, analyst overrides, case escalation, and feedback capture.



Figure 4. Pattern framing for ML-enabled human-in-the-loop fraud triage: context, forces, reusable solution, consequences, and related lineage. Synthesized from Lakshmanan et al. (2020), Andersen and Maalej (2024), Heiland et al. (2023), Cruz et al. (2023), and Järvenpää et al. (2024).

Figure 4 summarizes the pattern view. The relevant context is a fraud operation in which review demand exceeds analyst capacity, some cases require real-time action, attackers adapt, and error costs are asymmetric (Bolton and Hand, 2002; Carcillo et al., 2021; Ghadermazi et al., 2024; Jalalvand et al., 2024). The forces in tension include speed versus caution, customer friction versus missed fraud,

autonomy versus control, and expertise versus capacity (Alves et al., 2025; Bolton and Hand, 2002; Jalalvand et al., 2024). The reusable solution combines rules, scoring, policy, analyst review, and feedback loops, consistent with both the broader HIL-ML literature and more specific HIL pattern catalogues for ML-based systems (Andersen and Maalej, 2024; Mosqueira-Rey et al., 2023). The positive consequences include better use of scarce analyst attention, lower friction on clearly low-risk traffic, and stronger governance through an explicit separation between score and policy (Alves et al., 2025; Andersen and Maalej, 2024; Ghadermazi et al., 2024). The costs include architectural complexity, calibration work, queue-management overhead, and traceability or versioning obligations (Cruz et al., 2023; Lewis et al., 2021b; Nazir et al., 2024).

This framing turns the talk from a general exhortation into a reusable architectural idea: it clarifies when the pattern applies, why it is needed, and what trade-offs it carries. Forces and consequences do real work in that framing. They make trade-offs explicit in context, which is central to architectural reasoning (Richards and Ford, 2025).

5 The anatomy of the proposed pattern

The proposed pattern can be understood as a series of transformations that move from signal to action.

The first transformation goes from observation to score: the model consumes features and estimates risk. The second goes from score to policy: a separate policy layer interprets the score in light of operational goals and constraints. The third goes from policy to action, where the system approves, blocks, escalates, or routes a case to review (Figure 5).

Score $\neq$ decision: The model produces a risk estimate. A separate policy layer translates it into operational action under business constraints.

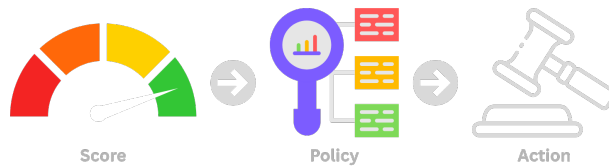
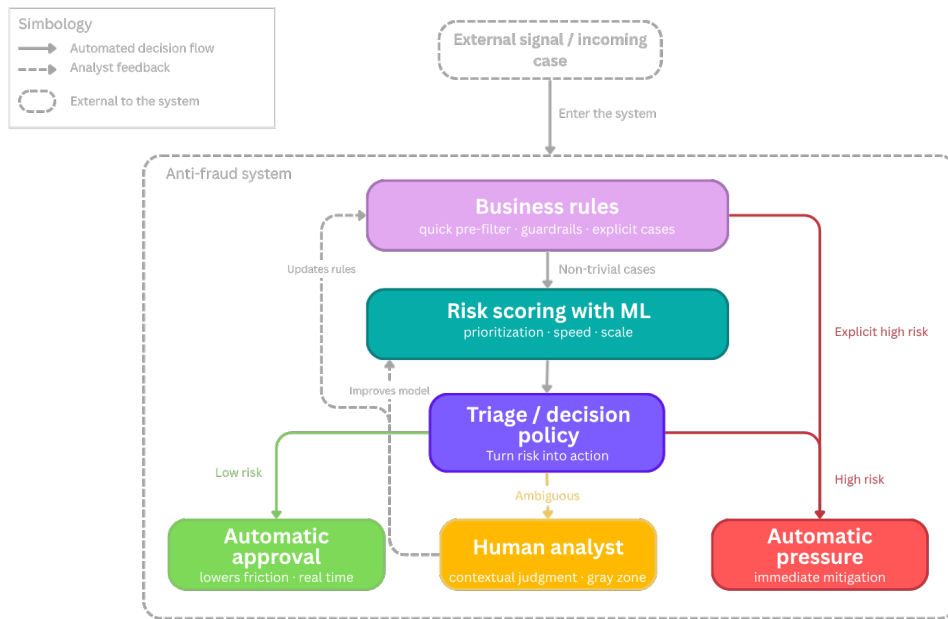


Figure 5. Score-to-policy-to-action pipeline: the model produces a risk score, a policy layer translates that score into operational logic, and only then does the system take action.

This separation matters because it blocks a common and dangerous shortcut: treating the model score as if it were already a business decision (Chen et al., 2022). In operational settings, scores often need to be combined with guardrails, analyst saturation, time sensitivity, customer value, regional policy, and legal requirements (Ghadermazi et al., 2024; Jalalvand et al., 2024; Kästner, 2025). Those are policy concerns, not model parameters.

Once score and policy are separated, the architecture can be stated more precisely. Figure 6 summarizes that runtime logic as a reference pattern: business

rules handle explicit cases, ML scores the non-trivial ones, and the policy layer decides whether the system should approve, escalate, or mitigate.



Learning to defer (L2D):

The system decides whether to handle a case automatically or route it to a human expert, based on who is more likely to be correct.

Figure 6. Reference architecture of the ML-enabled HIL Triage pattern: business rules filter explicit cases, ML scores the non-trivial cases, and a decision policy routes them to approval, analyst review, or automatic mitigation.

Deterministic rules sit near the entry point because some cases are explicit enough to justify immediate handling. Non-trivial cases reach the scoring layer, and the policy layer then converts score into action. Low-risk cases may be approved automatically to reduce friction. Clear high-risk cases may be mitigated automatically. Ambiguous cases move to analysts.

This structure has several advantages: it respects the strengths of each component, it separates inference from governance, and it creates a natural place to encode escalation logic. It also aligns well with the literature on learning to defer and selective intervention, where the task is deciding which cases stay with the model and which should be routed to experts, rather than classification alone (Alves et al., 2025).

6 Analysts are a runtime component

A recurring weakness in ML system design is to treat human review as a vague fallback step rather than as an explicit runtime component of the system (Andersen and Maalej, 2024; Ding et al., 2023; Mosqueira-Rey et al., 2023). The pattern proposed here argues for the opposite: analysts belong in the architecture as explicit runtime components.

Analyst as component:

Designed workflow with structured inputs (ranked cases, reason codes) and structured outputs (decisions, feedback).

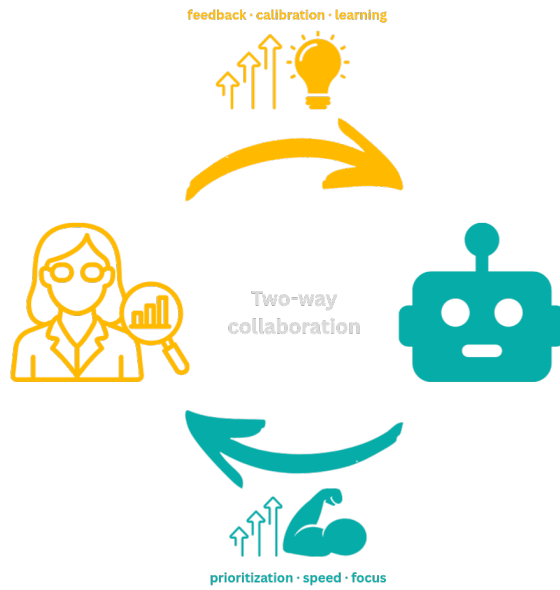


Figure 7. Bidirectional collaboration between analysts and ML: models provide prioritization, speed, and focus, while analysts contribute feedback, calibration, contextual judgment, and learning.

Figure 7 represents that relationship as a two-way collaboration. The model accelerates prioritization and reduces search cost. Analysts supply interpretation, exception handling, escalation judgment, and feedback. That feedback belongs to the production operation itself, not merely to an offline annotation step for training. Seen through the HIL literature, analysts may act as reviewers, collaborators, or teachers depending on where intervention happens and how their expertise is captured (Mosqueira-Rey et al., 2023). Related work on anomaly reasoning and management reaches a similar conclusion from the tooling side: beyond detection, production tooling should support explanation, action, and iterative investigation (Ding et al., 2023).

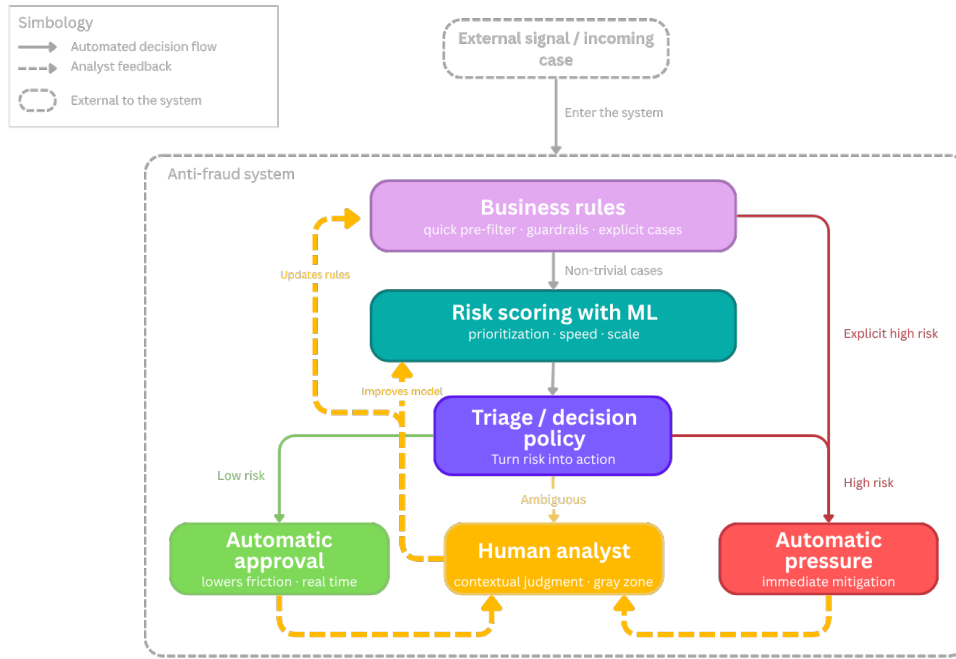


Figure 8. Closed-loop HIL triage with explicit feedback paths: analyst outcomes feed rule maintenance and model improvement, turning review into a learning mechanism rather than an operational dead end.

Analyst outcomes should feed both rule maintenance and model improvement, which means review results must be structured enough to support relabeling, rule creation, threshold changes, and post-incident analysis. Kadam (2024) sharpens this point by treating human-in-the-loop fraud feedback as something that can be propagated and reused, rather than as ad hoc review. That reuse becomes especially valuable when the system encounters weakly characterized or unknown attacks.

Expert-in-the-loop approaches to open-set recognition suggest that human review becomes most valuable at the boundary where the model faces novelty and uncertainty, rather than on well-understood cases (Yuan et al., 2026).

In Figure 8, the yellow return paths from automatic approval and automatic pressure to the human-analyst box make that point concrete. Analysts should review more than the ambiguous cases routed to them in real time. They should also be able to audit samples of cases that were automatically resolved on either side, both to detect false positives and false negatives and to decide whether the underlying label, threshold, rule, or routing logic should be revised. In production, that kind of retrospective review is valuable because many important errors become visible only after deployment, when the system encounters edge cases, data drift, or changing operating conditions and teams need explicit error monitoring rather than trust in aggregate metrics alone (Chen et al., 2022; Tan Wei Hao et al., 2026). Structured review of automatically handled cases can then be turned into corrections or additional labels, which Andersen and Maalej (2024) describe as a core HIL mechanism across operation

Feedback propagation:

Even sparse analyst annotations can improve fraud models when propagated through a transaction graph to related observations.

Open-set recognition:

Detecting cases outside any known class. Human review is most valuable at the boundary of model knowledge.

Retrospective audit:

Analysts should also review samples of automatically resolved cases to catch errors and revise routing logic.

and monitoring stages. In fraud settings, [Kadam \(2024\)](#) makes the same idea more concrete by showing that human feedback can be propagated and reused, improving robustness, recall, and performance on unseen fraud patterns rather than remaining a one-off operational intervention. When those reviewed outcomes are captured carefully, they also improve relabeling quality and strengthen future training data, especially in settings where only a subset of predictions can be manually verified ([Chen et al., 2022](#)).

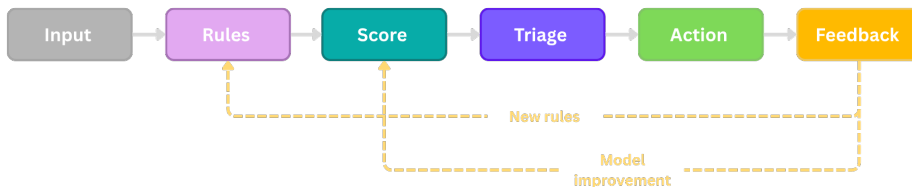
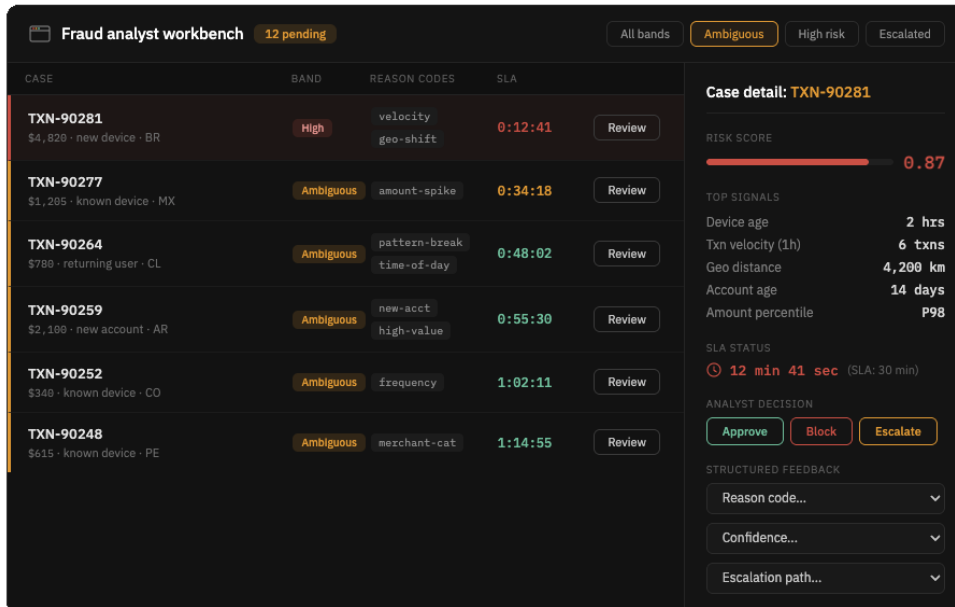


Figure 9. Operative learning in ML-enabled HIL triage: downstream feedback should update both rules and models so that operations become a source of system learning.

At its simplest, the pattern is an operating loop rather than a pipeline (Figure 9). The system should not end with action alone. Once outcomes are observed, feedback should flow back into both rules and models so that operations become a source of learning.

Operationalization also requires a concrete analyst workbench.



Analyst workbench:
Ranked cases, risk bands, reason codes, SLA timers, action controls, structured feedback fields.

Figure 10. Analyst workbench for HIL fraud triage: ranked cases, risk bands, reason codes, SLA timers, and structured feedback turn the architecture into an operational review system. Synthesized from [Jalalvand et al. \(2024\)](#), [Ghadermazi et al. \(2024\)](#), and [Alves et al. \(2025\)](#).

A workbench like the one shown in Figure 10 turns the architecture into an operational review system. Alert prioritization research repeatedly highlights the importance of workload, context, skill, assignment, and review efficiency (Ghadermazi et al., 2024; Jalalvand et al., 2024). Learning-to-defer research likewise shows that expert availability and heterogeneity matter for system performance (Alves et al., 2025). A practical analyst interface should therefore expose ranked cases, top signals, SLA pressure, action controls, and structured feedback fields; without those elements, the architecture stays abstract and hard to operate. This is also consistent with human-AI interaction guidance, which emphasizes communicating uncertainty, supporting efficient oversight, and making intervention understandable at the point of use (Amershi et al., 2019b). Industry case studies point in the same direction. Uber’s Project RADAR used humans in the loop to validate and operationalize candidate fraud rules rather than treating analysts as a purely manual backup layer (Zelvenskiy et al., 2022).

7 Evaluation must fit the socio-technical system

One of the strongest consequences of the architecture view is that evaluation has to widen (Lewis et al., 2021b; Nazir et al., 2024). A system that routes decisions across automation and human review should not be judged with a single model metric, because the relevant unit of evaluation is the broader socio-technical system in which model outputs interact with workflow, human interpretation, and organizational action (Dobbe and Wolters, 2024). ML is one element inside a larger work system, and evaluating the ML component in isolation misses the interactions that determine real outcomes (Salwei and Carayon, 2022).

Four evaluation layers:
(1) Data/input quality,
(2) model/decision
quality, (3) queue/analyst
operations,
(4) runtime/platform
quality. Plus policy
behavior.

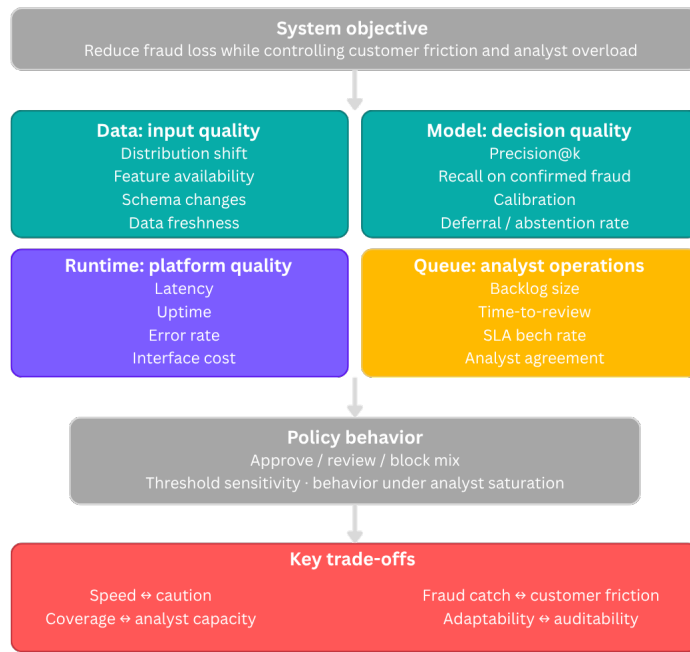


Figure 11. Metrics and trade-offs for ML-enabled human-in-the-loop fraud triage: model quality, queue operations, runtime quality, and policy behavior should be evaluated together. Synthesized from Lewis et al. (2021b), Jalalvand et al. (2024), Ghadermazi et al. (2024), Alves et al. (2025), Chen et al. (2022), and Tan Wei Hao et al. (2026).

These dimensions define a broader evaluation frame for the system (Figure 11), one that reflects both model behavior and the wider socio-technical conditions under which the system is used and interpreted (Dobbe and Wolters, 2024; Fazelpour, 2025). Data and input quality matter because distribution shifts, missing features, or stale inputs can damage performance before the model even acts (Chen et al., 2022; Kästner, 2025; Lewis et al., 2021b). Model and decision quality still matter, but metrics such as precision at the top of the queue (precision@k), recall on confirmed fraud, calibration, and deferral rate are often more informative than a single aggregate score (Alves et al., 2025; Chen et al., 2022).

Queue and analyst operations matter because backlog size, review latency, agreement, workload, and SLA behavior shape the system’s real effectiveness in practice (Alves et al., 2025; Ghadermazi et al., 2024; Jalalvand et al., 2024). Runtime and platform quality matter because latency, error rates, uptime, and observability determine whether the decision system remains usable in production (Lewis et al., 2021b; Tan Wei Hao et al., 2026).

The policy box in Figure 11 is worth pausing on. Approval, review, and block rates are more than downstream consequences of the model. They are part of what the organization deliberately chooses through its operating policy, because predictions still have to be translated into product behavior, business logic, and action (Chen et al., 2022). This is another reason to keep score

Precision@k: Precision among the top- k ranked cases. More relevant when analyst capacity is finite.

Deferral rate: Fraction of cases routed to human review. Key tuning parameter for workload.

Policy behavior: The approve/review/block split is an organizational choice, not just a model output. Threshold sensitivity and saturation behavior are policy metrics.

and policy separate: the model estimates risk, but the organization decides how that risk will be acted on under its own constraints, quality goals, and operating conditions (Kästner, 2025; Lewis et al., 2021b).

The trade-off strip at the bottom of the figure summarizes a real constraint: fraud systems move along architectural and operational frontiers rather than optimizing a single objective, and the meaning of those trade-offs depends on the broader system in which the model is embedded (Fazelpour, 2025). Speed can conflict with caution, fraud catch rate with customer friction, coverage with analyst capacity, and adaptability with auditability (Alves et al., 2025; Chen et al., 2022; Jalalvand et al., 2024). More broadly, architecting ML-enabled systems requires making heterogeneous quality concerns, uncertainty, and decision trade-offs explicit rather than pretending they disappear at deployment time (Cruz et al., 2023; Kästner, 2025; Lewis et al., 2021b; Nazir et al., 2024; Richards and Ford, 2025).

8 MLOps and platform engineering are part of the argument

The pattern does not end at runtime routing. It has an operational lifecycle in which monitoring and user feedback inform model maintenance and evolution, including retraining (Lewis et al., 2021b). In production, those same signals should also support data curation, feature maintenance, and policy revision rather than being treated as isolated operational artifacts (Kästner, 2025; Tan Wei Hao et al., 2026). Figure 12 summarizes that broader lifecycle, linking offline build activities with online operation, monitoring, feedback capture, and iterative revision. The upper part of the figure represents the offline build loop of data capture, feature generation, training, and evaluation. The lower part represents the online operating loop, where deployment, monitoring, analyst review, and structured feedback create the conditions for maintenance and adaptation.

MLOps lifecycle: Data
 → features → training →
 eval → deploy → monitor
 → review → feedback.
 Three return loops: data
 curation, model
 improvement, policy
 revision.

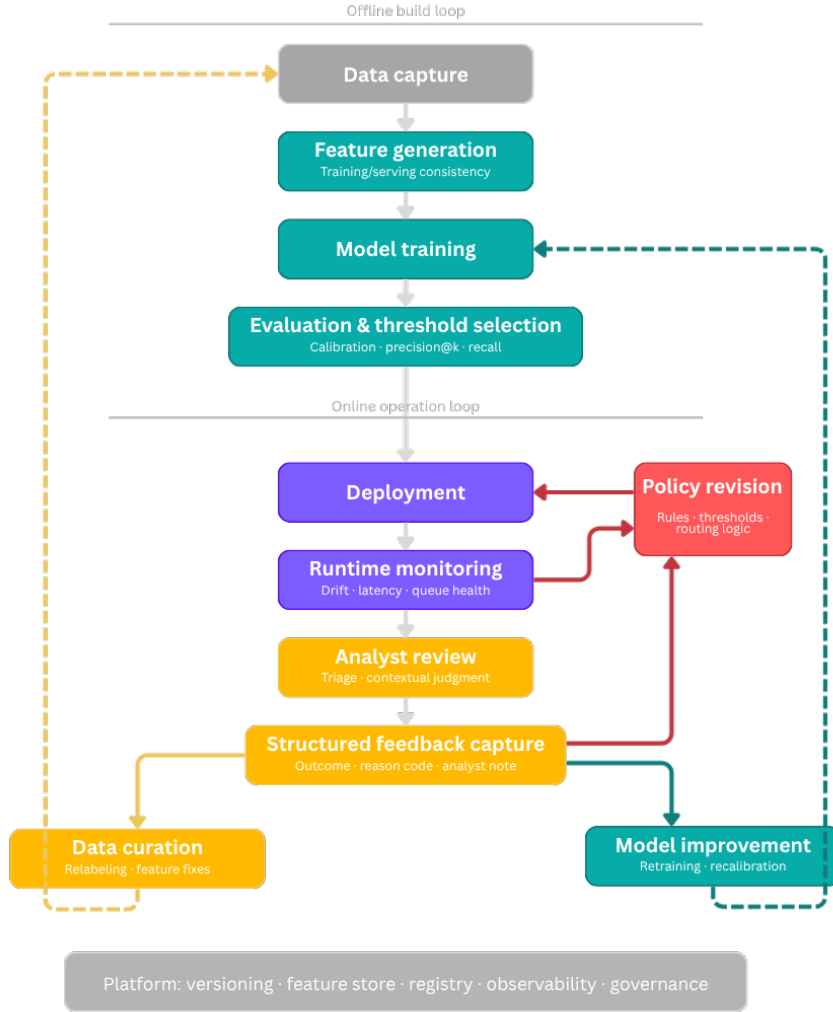


Figure 12. MLOps feedback lifecycle for ML-enabled HIL fraud systems: runtime monitoring and analyst feedback support data curation, model improvement, and policy revision. Adapted from Lewis et al. (2021b), extended with platform-engineering concepts from Tan Wei Hao et al. (2026), and specialized for fraud feedback loops using Kadam (2024).

The side loops are the most important part of the diagram, because they show why feedback should not collapse into a single retraining arrow. Some signals should improve the dataset or feature pipeline, some should drive model improvement, and some should revise rules, thresholds, or routing policy. Data and feature plumbing belong to the same architectural story. Lin and Ryaboy (2013) describe this broader production problem as one of workflows, recurring jobs, and infrastructure plumbing, where value depends not only on mining algorithms but on the systems that make large-scale analysis repeatable and operable. Konieczny (2025) documents recurring data-engineering patterns for dependable ingestion, transformation, lineage, and pipeline quality, and those concerns map directly onto the data-capture and feature-generation stages shown here. Mosqueira-Rey et al. (2023) help explain why the diagram contains more than one return path: in HIL systems, human intervention may serve

different roles, from annotation to interactive steering to teaching, so collapsing all feedback into retraining would hide important design distinctions.

This is consistent with the literature. [Lewis et al. \(2021b\)](#) emphasize monitorability, maintenance, and evolution. [Lewis et al. \(2021a\)](#) discuss mismatch between assumptions made by different roles and system parts. [Tan Wei Hao et al. \(2026\)](#) frame platform support as an important way to reduce accidental complexity and create repeatable paths for deployment, observability, and governance. [Kadam \(2024\)](#) brings the fraud-specific feedback loop into that broader lifecycle.

The platform foundation shown in Figure 12 also matters. Versioning, feature storage, registry services, observability, and governance are not optional conveniences for mature systems. They are part of what makes the architecture maintainable.

Monitorability:

First-class quality attribute for ML systems. Separate monitoring for data quality, model quality, service health, and queue health.

ML platform: Versioning, feature store, model registry, observability, governance. Reduces accidental complexity across teams.

9 What the pattern improves, and what it costs

The proposed pattern improves several things at once. It uses analyst attention more efficiently by reserving human review for the cases where human judgment creates the most value, consistent with alert-prioritization and learning-to-defer work on workload, expertise, and capacity constraints ([Alves et al., 2025](#); [Jalalvand et al., 2024](#)). It reduces friction on clearly low-risk traffic while making it easier to intervene quickly on clear high-risk cases, which is one reason prioritization and routing logic matter in operational systems and not only in model evaluation ([Jalalvand et al., 2024](#); [Kästner, 2025](#)). It separates inference from governance by giving policy its own architectural place, instead of treating model output as if it were already the organization's decision ([Chen et al., 2022](#); [Lewis et al., 2021b](#)). And it makes learning more explicit by turning review outcomes into inputs for rule and model improvement, aligning the runtime system with broader HIL patterns of feedback, correction, and continued adaptation ([Andersen and Maalej, 2024](#); [Mosqueira-Rey et al., 2023](#)).

Those benefits come with costs. The architecture has more moving parts than a pure scoring service, and ML-enabled systems already create substantial integration, data, and evolution challenges even before the human lane is added ([Lewis et al., 2021b](#); [Nazir et al., 2024](#)). Queue behavior must be monitored, because backlog, latency, analyst workload, and assignment quality shape the real system outcome, not just the model's score distribution ([Ghadermazi et al., 2024](#); [Jalalvand et al., 2024](#)). Decision logic must be versioned and auditable, and human feedback must be structured and governed, because traceability, co-versioning, and explicit decision rationale are part of reliable ML-enabled architecture rather than optional extras ([Cruz et al., 2023](#); [Lewis et al., 2021b](#)). Different teams (software engineers, data scientists, risk operators, analysts) must also coordinate around shared system behavior, exactly the kind of trade-off-rich architectural work emphasized in both ML-enabled systems research

and general architecture literature (Nazir et al., 2024; Richards and Ford, 2025). The design-pattern framing earns its keep here, since it documents the costs of adoption alongside the solution.

9.1 Anti-patterns worth naming

Several anti-patterns follow naturally from this discussion.

Letting the score become policy. When a raw model score directly triggers business action without an explicit policy layer, governance becomes opaque: the system no longer makes clear where inference ends and organizational decision-making begins. A separate policy layer is useful precisely because ML-enabled systems need architectural separation of concerns rather than tight coupling between business logic and inference (Lewis et al., 2021b; Washizaki et al., 2020). Threshold changes also become risky in tightly coupled designs, since teams lose the ability to distinguish whether a failure belongs to the model, the surrounding business logic, or the interaction between both parts (Washizaki et al., 2020). The remedy is the separation described in Section 5: score estimates risk; policy decides what to do about it. That separation is consistent with the recurring ML architecture pattern of distinguishing business logic from ML models, and with product-oriented ML engineering that treats prediction as only one step inside a larger decision system (Kästner, 2025; Washizaki et al., 2020).

Anti-pattern, Score = policy: Raw scores trigger actions; governance becomes opaque.

Treating analysts as unstructured exception handling. If reviewers leave only free-text notes and the system captures no structured outcome data, the feedback loop collapses. In HIL systems, human intervention can play very different roles, including annotation, interactive steering, and machine teaching, so the architecture needs to capture that intervention explicitly rather than leaving it implicit in notes (Mosqueira-Rey et al., 2023). The review lane then generates operational cost but no reusable signal. HIL pattern work, by contrast, treats human feedback as an explicit part of operation and monitoring instead of an informal fallback around an otherwise autonomous model (Andersen and Maalej, 2024). Structured fields for outcome, reason code, confidence, and suggested action are what turn review into learning: they make the analyst contribution reusable for correction, adaptation, and future improvement rather than leaving it trapped inside case-by-case exception handling (Andersen and Maalej, 2024; Mosqueira-Rey et al., 2023).

Anti-pattern, Unstructured HIL: Free-text-only review generates cost but no reusable signal.

Building the model while leaving operations underspecified. A common failure mode is to invest heavily in data science while treating the analyst interface, queue design, and escalation logic as afterthoughts. Alert-prioritization research repeatedly shows that increasing alert volume overwhelms analysts and creates alert fatigue, so operational design is not peripheral to system performance (Jalalvand et al., 2024). Workload, time efficiency, analyst fatigue, skill, and contextual review conditions all affect how well the system works in practice, not just how well the model performs in isolation (Jalalvand et al., 2024). Underspecifying operations therefore yields a high-performing model

inside a system that cannot operate it effectively. More broadly, ML-enabled systems require architecture work that covers both the model pipelines and the system that uses and interacts with ML components, especially for maintenance, evolution, and coordination across teams (Lewis et al., 2021b; Nazir et al., 2024).

Ignoring monitorability until something breaks. Lewis et al. (2021b) identify monitorability as a top architectural driver for the maintenance and evolution of ML systems, and they note that ML systems need end-to-end lifecycle support that includes monitoring, maintenance, and evolution. In fraud systems, degradation often appears first as queue overload, rising analyst disagreement, or calibration drift rather than as a dramatic crash, so waiting for obvious failure is itself an anti-pattern (Jalalvand et al., 2024; Lewis et al., 2021b).

Glue code and pipeline jungles. Washizaki et al. (2020) identify both glue code and pipeline jungles as recurring ML anti-patterns. Fraud systems are especially vulnerable because they accumulate feature joins, rule engines, risk services, case queues, and alerting layers under delivery pressure. Without platform discipline, traceability, and versioning, those components become fragile and hard to change (Kästner, 2025; Lewis et al., 2021b; Washizaki et al., 2020).

Anti-pattern, Pipeline jungle: Feature joins, rule engines, risk services accumulate without platform discipline.

These anti-patterns define the limits of the pattern as clearly as its benefits. The architecture works only if score and policy remain distinct, feedback is structured, and operations are designed and monitored explicitly rather than left implicit in ad hoc practice (Andersen and Maalej, 2024; Lewis et al., 2021b; Washizaki et al., 2020). Their value goes beyond diagnosis, too: they show why the pattern belongs to software architecture in the first place, as a way of making system responsibilities, trade-offs, and failure modes visible across software, data, policy, and human work (Cruz et al., 2023; Richards and Ford, 2025). That broader architectural view is what the next section builds on.

10 Why this matters for a new, but important field

A recurring idea across the literature discussed here is that ML-enabled software architecture is still a relatively young field. Muccini and Vaidhyanathan (2021) explicitly argue that the community still needs better architecting practices and more clearly defined standards for ML-based software systems. Nazir et al. (2024) reinforce that point by showing that the field is still actively consolidating design challenges, best practices, and architectural design decisions. The community already has important concepts, such as hidden technical debt, architecture mismatch, HIL design patterns, monitorability, architecture evaluation, and internal ML platforms, but reusable design knowledge is still being consolidated. At the same time, the wider HIL-ML literature makes clear that human participation is itself a design space rather than a generic

fallback (Andersen and Maalej, 2024; Mosqueira-Rey et al., 2023).

Fraud prevention is a useful place to push that field forward because the stakes make the socio-technical nature of ML especially visible. Dobbe and Wolters (2024) argue that ML applications should be understood not only in an algorithmic or data frame, but in a sociotechnical frame that includes human and institutional decisions within the system boundary. Salwei and Carayon (2022) similarly stress that AI is only one element of a larger work system, and that design has to account for workflow, users, technologies, and organizational context together. Kästner (2025) reinforces the same point from a production-engineering perspective by treating machine learning as part of a product and operational system rather than as an isolated model. In lower-stakes domains, teams can sometimes hide the surrounding architecture behind a model score or a dashboard. Fraud operations expose that move very quickly: decisions affect money, customer experience, investigation load, escalation paths, and organizational risk. That makes fraud a particularly revealing domain for discussing ML-enabled architecture as architecture.

The design-pattern proposition developed here is meant to add one more concrete example to that growing field. We claim no novelty for humans in the loop as such. What we do claim is that this particular combination of rules, scoring, policy, analyst review, and closed-loop feedback deserves to be documented as a reusable architectural response.

That claim draws support from three directions. First, the pattern literature for ML-based systems has been growing steadily, and documenting reusable solutions is exactly what pattern work is for (Andersen and Maalej, 2024; Washizaki et al., 2020). Second, production-oriented ML engineering increasingly treats models as components inside larger products and operational systems rather than as standalone artifacts (Kästner, 2025); a fraud model that works well on a test set but lacks surrounding architecture for routing, feedback, and governance is not yet a production system. Third, software architecture as a discipline is built on making responsibilities and trade-offs explicit (Richards and Ford, 2025), and the pattern proposed here follows that discipline: it names the components, describes their interactions, and documents the costs of adopting them.

11 Open questions

Several questions remain open, and five are worth stating explicitly.

The first concerns policy optimization. If scores, analyst capacity, SLA pressure, and business value all matter, what is the best way to define and adapt the routing policy over time?

The second concerns feedback quality. Which analyst signals are most predictive of future system improvement, and how should disagreement between analysts be modeled? Learning-to-defer research suggests that expert heterogeneity

and capacity constraints matter (Alves et al., 2025); in fraud operations, those effects are likely to interact further with role-specific responsibilities and queue dynamics, which deserve more direct study. Expert-in-the-loop work on unknown attack detection via open-set recognition points to one promising direction, but similar ideas remain underexplored in fraud operations (Yuan et al., 2026).

The third concerns architecture evaluation. Scenario-based architecture evaluation has already been discussed for ML-enabled systems (Cruz et al., 2023; Nazir et al., 2024), but there is still room for domain-specific methods tailored to operational fraud settings. Useful evaluation scenarios in this domain would go past generic quality attributes and explicitly test queue saturation, feature outages, delayed labels, and adversarial shifts, because those are the conditions that determine whether a fraud architecture remains effective after deployment (Chen et al., 2022; Jalalvand et al., 2024; Lunghi et al., 2023).

The fourth concerns platform productization. Many organizations still build fraud systems as collections of scripts, dashboards, and services with weak shared abstractions, which makes cross-team coordination, workflow orchestration, and operational visibility harder than they need to be. Lin and Ryaboy (2013) describe this broader “plumbing” problem in large-scale data systems, where workflows span teams and frameworks in ways that become difficult to reason about without stronger shared infrastructure. In ML systems, Tan Wei Hao et al. (2026) argue that well-designed internal platforms can provide a more consistent foundation for automation, monitoring, deployment, and developer experience, while Kästner (2025) emphasizes that orchestration, observability, provenance, and infrastructure discipline are part of the product system rather than secondary implementation details. Internal platform engineering for fraud and risk therefore remains an important opportunity.

The fifth concerns large language models and generative AI in the analyst workflow. LLM-based components may eventually change how analysts interact with fraud systems by supporting case summarization, natural-language reasoning traces, conversational investigation support, or explanation interfaces. Tan Wei Hao et al. (2026) show that LLM-powered systems introduce distinctive concerns around interface design, validation, observability, safety guardrails, and evaluation, which means they should not be bolted on as thin interface layers without architectural planning. In fraud operations, the pressing question is how to integrate these components without weakening the structured feedback loops, auditability, and governance on which the proposed pattern depends. At this stage, that question remains a design agenda rather than a settled answer.

LLM integration: How to add NL case summarization and conversational investigation without undermining structured feedback and auditability.

12 Conclusion

We argued that fraud prevention should be designed as an ML-enabled socio-technical architecture. System behavior is produced by the interaction of rules,

machine learning, analyst work, policy, and platform support, not by the model alone. Robust design therefore requires more than a strong classifier: explicit separation between score and policy, deliberate routing of ambiguous cases to expert review, structured feedback loops, evaluation that extends beyond accuracy, and operational practices that connect runtime behavior back to data, models, and policy.

The proposed pattern can be stated simply. Use rules where the organization already has explicit knowledge and firm guardrails. Use machine learning where prioritization and signal compression create scale. Use analysts where context, exception handling, and judgment matter. Keep score separate from policy. Capture feedback in a structured form. Design the surrounding platform so that the system can be observed, revised, and improved over time.

“Keep humans in the loop” is only half the message. The other half is that this specific combination of rules, scoring, policy, analyst review, and closed-loop learning can be treated as a reusable architectural response for a high-stakes domain. That is the argument presented in the talk and developed in greater depth here.

References

- Alves, J. a. V., Leitão, D., Jesus, S., Sampaio, M. O. P., Liébana, J., Saleiro, P., Figueiredo, M. A. T., and Bizarro, P. (2025). A benchmarking framework and dataset for learning to defer in human-AI decision-making. *Scientific Data*, 12:506.
- Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., and Nushi, B. (2019a). Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 291–300. IEEE.
- Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S. T., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., and Horvitz, E. (2019b). Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery.
- Andersen, J. and Maalej, W. (2024). Design patterns for machine learning-based systems with humans in the loop. *IEEE Software*, 41(4):151–159.
- Bolton, R. J. and Hand, D. J. (2002). Statistical fraud detection: A review. *Statistical Science*, 17(3):235–255.
- Carcillo, F., Le Borgne, Y.-A., Caelen, O., Kessaci, Y., Oblé, F., and Bontempi, G. (2021). Combining unsupervised and supervised learning in credit card fraud detection. *Information Sciences*, 557:317–331.

- Chen, C., Murphy, N. R., Parisa, K., Sculley, D., and Underwood, T. (2022). *Reliable Machine Learning: Applying SRE Principles to ML in Production*. O'Reilly Media.
- Cruz, P., Ulloa, G., San Martin, D., and Veloz, A. (2023). Software architecture evaluation of a machine learning enabled system: A case study. In *2023 42nd IEEE International Conference of the Chilean Computer Science Society (SCCC)*. IEEE.
- Ding, X., Seleznev, N., Kumar, S., Bruss, C. B., and Akoglu, L. (2023). From detection to action: A human-in-the-loop toolkit for anomaly reasoning and management. In *Proceedings of the Fourth ACM International Conference on AI in Finance*, pages 279–287. Association for Computing Machinery.
- Dobbe, R. and Wolters, A. (2024). Toward sociotechnical AI: Mapping vulnerabilities for machine learning in context. *Minds and Machines*, 34(2):Article 12.
- Fazelpour, S. (2025). Disciplining deliberation: A socio-technical perspective on machine learning trade-offs. *The British Journal for the Philosophy of Science*. Advance online publication.
- Ghadermazi, J., Shah, A., and Jajodia, S. (2024). A machine learning and optimization framework for efficient alert management in a cybersecurity operations center. *Digital Threats: Research and Practice*, 5(2):Article 19.
- Heiland, L., Hauser, M., and Bogner, J. (2023). Design patterns for AI-based systems: A multivocal literature review and pattern repository. In *2023 IEEE/ACM 2nd International Conference on AI Engineering—Software Engineering for AI (CAIN)*. IEEE.
- Hernandez Aros, L., Bustamante Molano, L. X., Gutierrez-Portela, F., Moreno Hernandez, J. J., and Rodríguez Barrero, M. S. (2024). Financial fraud detection through the application of machine learning techniques: A literature review. *Humanities and Social Sciences Communications*, 11(1):1–22.
- Hilal, W., Gadsden, S. A., and Yawney, J. (2022). Financial fraud: A review of anomaly detection techniques and recent advances. *Expert Systems with Applications*, 193:116429.
- Jalalvand, F., Chhetri, M. B., Nepal, S., and Paris, C. (2024). Alert prioritisation in security operations centres: A systematic survey on criteria and methods. *ACM Computing Surveys*, 57(2):Article 42.
- Järvenpää, H., Lago, P., Bogner, J., Lewis, G. A., Muccini, H., and Ozkaya, I. (2024). A synthesis of green architectural tactics for ML-enabled systems. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS '24)*, pages 130–141. Association for Computing Machinery.

- Kadam, P. (2024). Enhancing financial fraud detection with human-in-the-loop feedback and feedback propagation. In *2024 International Conference on Machine Learning and Applications (ICMLA)*, pages 1198–1203. IEEE.
- Kästner, C. (2025). *Machine Learning in Production: From Models to Products*. MIT Press.
- Konieczny, B. (2025). *Data Engineering Design Patterns: Recipes for Solving the Most Common Data Engineering Problems*. O’Reilly Media.
- Lakshmanan, V., Robinson, S., and Munn, M. (2020). *Machine Learning Design Patterns*. O’Reilly Media.
- Lewis, G. A., Bellomo, S., and Ozkaya, I. (2021a). Characterizing and detecting mismatch in machine-learning-enabled systems. In *2021 IEEE/ACM 1st Workshop on AI Engineering—Software Engineering for AI (WAIN)*, pages 133–140. IEEE.
- Lewis, G. A., Ozkaya, I., and Xu, X. (2021b). Software architecture challenges for ML systems. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 634–638. IEEE.
- Lin, J. and Ryaboy, D. (2013). Scaling big data mining infrastructure: The Twitter experience. *ACM SIGKDD Explorations Newsletter*, 14(2):6–19.
- Lunghi, D., Simitsis, A., Caelen, O., and Bontempi, G. (2023). Adversarial learning in real-world fraud detection: Challenges and perspectives. In *Proceedings of the Second ACM Data Economy Workshop*, pages 27–33. Association for Computing Machinery.
- Mosqueira-Rey, E., Hernández-Pereira, E., Alonso-Ríos, D., Bobes-Bascarán, J., and Fernández-Leal, A. (2023). Human-in-the-loop machine learning: A state of the art. *Artificial Intelligence Review*, 56:3005–3054.
- Muccini, H. and Vaidhyathan, K. (2021). Software architecture for ML-based systems: What exists and what lies ahead. In *2021 IEEE/ACM 1st Workshop on AI Engineering—Software Engineering for AI (WAIN)*, pages 121–128. IEEE.
- Nazir, R., Bucaioni, A., and Pelliccione, P. (2024). Architecting ML-enabled systems: Challenges, best practices, and design decisions. *Journal of Systems and Software*, 207:111860.
- Richards, M. and Ford, N. (2025). *Fundamentals of Software Architecture: A Modern Engineering Approach*. O’Reilly Media, 2nd edition.
- Salwei, M. E. and Carayon, P. (2022). A sociotechnical systems framework for the application of artificial intelligence in health care delivery. *Journal of Cognitive Engineering and Decision Making*, 16(4):194–206.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hid-

den technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, volume 28.

Tan Wei Hao, B., Padmanabhan, S., and Mallya, V. (2026). *Machine Learning Platform Engineering: Build an Internal Developer Platform for ML and AI Systems*. Manning.

Washizaki, H., Uchida, H., Khomh, F., and Guéhéneuc, Y.-G. (2020). Machine learning architecture and design patterns. *IEEE Software*, 37(4):76–84.

Yuan, X., Yu, P., Liu, S., Sun, Z., Zhang, Y., and Xu, J. (2026). An expert-in-the-loop framework for unknown attack detection via open-set recognition. *Journal of Computer Security*. Advance online publication.

Zelvenskiy, S., Harisinghani, G., Yu, T., Ng, E., and Wei, R. (2022). Project RADAR: Intelligent early fraud detection system with humans in the loop. Uber Blog.